

Penetration Testing Report

OWASP Top 10 Practical Assessment — DVWA (Damn Vulnerable Web Application)

Skill Set Go EduTech — Cybersecurity Internship

Week 2 — Ethical Hacking & Web Security

Prepared by: Muhammad Usman • Date: September 2026

Classification: Internal Training / Legal Practice-Environment Assessment

1. Executive Summary

This report documents a black-box, authenticated web application security assessment performed against DVWA (Damn Vulnerable Web Application), a deliberately vulnerable training platform, running locally in a controlled Docker environment. The objective was to identify and document real vulnerabilities aligned with the OWASP Top 10:2025 framework, as part of Week 2 of a structured cybersecurity internship.

Three vulnerabilities were identified and validated: a classic SQL Injection in a user-lookup form, an OS Command Injection in a network utility feature, and a predictable session-identifier generation scheme. Two of the three findings are rated Critical or High severity, both stemming from the same root cause — untrusted user input reaching a sensitive interpreter (SQL engine or system shell) without validation.

Finding	OWASP 2025 Category	Severity
SQL Injection (User ID lookup)	A05:2025 — Injection	High
OS Command Injection (ping utility)	A05:2025 — Injection	Critical
Predictable Session ID Generation	A07:2025 — Authentication Failures / A04:2025 — Cryptographic Failures	High

2. Scope & Methodology

2.1 Scope

- **Target application:** DVWA (Damn Vulnerable Web Application) v1.10 (Development build)
- **Environment:** Local Docker container, `http://localhost:8080`
- **Security level tested:** Low (to reliably demonstrate exploitability) and High (for session-ID source review)
- **Access level:** Authenticated, default credentials (admin / password)
- **Out of scope:** Denial-of-service testing; any system other than the local DVWA container

2.2 Methodology

- **Reconnaissance** — identified the application, version, and available vulnerability modules.
- **Manual input testing** — submitted crafted payloads to input fields (User ID, IP address) to test how untrusted input was handled.
- **Source code review** — inspected server-side PHP source (available via DVWA's built-in source viewer) to confirm root cause of the session-ID weakness rather than relying on observed behavior alone.
- **Evidence capture** — screenshotted each successful test and the relevant application/server response.
- **Risk rating** — rated each finding by likelihood and impact, and mapped it to the OWASP Top 10:2025 classification.

3. Detailed Findings

3.1 SQL Injection — User ID Lookup

Severity: High **OWASP 2025:** A05:2025 — Injection

Location: SQL Injection module, User ID input field

Description: The User ID field, intended to look up a single user record, passes input directly into a SQL query without parameterization or sanitization.

Payload submitted: `%'` or `'1'='1`

Result: The query returned all 5 user records in the database (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) instead of a single matching record, confirming the injected boolean condition (`'1'='1'`) was evaluated as part of the SQL WHERE clause.

Impact: An attacker can enumerate the entire user table and, depending on query structure elsewhere in the application, potentially extend this into extracting additional data via UNION-based injection (e.g. stored password hashes).

Remediation: Replace string-concatenated queries with parameterized queries or prepared statements. Apply the principle of least privilege to the database account used by the application.

3.2 OS Command Injection — Ping Utility

Severity: Critical **OWASP 2025:** A05:2025 — Injection

Location: Command Injection module, IP address input field

Description: The IP address field is passed to a system shell command (ping) without validating that the input is a well-formed IP address or stripping shell metacharacters.

Payload submitted: 127.0.0.1 && whoami

Result: The application returned the expected ping output for 127.0.0.1, followed by an additional unrequested line: www-data — the output of the injected whoami command, confirming the web server process user.

Impact: Full arbitrary OS command execution as the web server user. This is the most severe finding in this assessment — depending on further privilege boundaries, an attacker could read sensitive files, establish persistence, or pivot to other systems.

Remediation: Avoid passing user input to shell commands. Where unavoidable, strictly validate input against an allow-list pattern (e.g. a regex matching valid IPv4/IPv6 syntax) and use APIs that execute commands with arguments passed as an array, avoiding shell interpretation entirely.

3.3 Predictable Session ID Generation

Severity: High **OWASP 2025:** A07:2025 — Authentication Failures / A04:2025 — Cryptographic Failures

Location: Weak Session IDs module, server-side session cookie generation logic (reviewed via source code, High security level)

Description: Server-side PHP source confirms the session cookie value is derived from an MD5 hash of a simple incrementing integer counter, with no random component:

```
$_SESSION['last_session_id_high']++;  
$cookie_value = md5($_SESSION['last_session_id_high']);
```

Result: Since MD5 is a fast, publicly known, unkeyed hash function, and the only input is a small sequential integer, an attacker who observes one valid session value can pre-compute the MD5 hash of nearby integers to generate a list of likely-valid session identifiers for other users.

Impact: Session hijacking without needing a victim's password — an attacker who guesses/predicts a valid session ID gains the associated user's authenticated session outright.

Remediation: Generate session identifiers using a cryptographically secure random source (e.g. PHP's random_bytes()), never a hash of predictable input. Ensure adequate token length/entropy, and set HttpOnly, Secure, and SameSite flags on session cookies.

4. Risk Summary

#	Finding	Likelihood	Impact	Overall Risk
1	SQL Injection	High	High	High
2	OS Command Injection	High	Critical	Critical
3	Predictable Session IDs	High	High	High

5. Prioritized Recommendations

Priority	Recommendation	Addresses
1	Use parameterized queries / an ORM for all database access	Finding 1
2	Eliminate shell command construction from user input; use allow-listed, argument-array execution	Finding 2
3	Replace hash-of-counter session IDs with cryptographically secure random tokens	Finding 3
4	Set HttpOnly / Secure / SameSite flags on all session cookies	Finding 3

6. Conclusion

This assessment identified three vulnerabilities in DVWA — one Critical and two High severity — spanning two OWASP Top 10:2025 categories (Injection; Authentication Failures / Cryptographic Failures). All three findings trace back to a common underlying pattern: untrusted input reaching a sensitive interpreter (a database engine, a system shell, or a hashing function) without adequate validation, encoding, or randomness. Remediating the Command Injection finding should be treated as the immediate priority given its Critical rating and full remote-code-execution potential, followed by the SQL Injection and session-ID weaknesses.

This report was produced as part of Week 2 of the Skill Set Go Cybersecurity Internship, building on Week 1's assessment of OWASP Juice Shop (which covered Injection and Security Misconfiguration). Together, the two assessments now demonstrate validated findings across four distinct OWASP Top 10:2025 categories.

— End of Report —